

Vulnerability Assessment Report

Altoro Mutual — demo.testfire.net

Assessment type	Read-only, passive external
Date	September 2026
Submitted by	Preetam Karoshi, Cyber Security Intern

9 findings • 3 High • 4 Medium • 1 Low • 1 Info

This report covers a public training/demo environment only — no production systems, customer data, or real financial infrastructure were involved. Written as part of a cyber security internship assignment.

What I found, in plain terms

For this task, I ran a read-only, passive security review of Altoro Mutual's public-facing website — a demo banking app IBM/HCL publish specifically for people to practice this kind of assessment on. This was my first hands-on vulnerability assessment, so I kept strictly to the passive side of things: no login attempts, no exploitation, no brute-forcing, nothing that could disrupt the site.

I ended up with **9 findings** across four risk levels. I didn't try to exploit or prove any of the High findings myself — two are configuration mistakes with a fairly direct fix, and one is a pattern I noticed but deliberately didn't test further, since confirming it would mean actively probing the target.

MY TAKEAWAY

The site's biggest weak spots are around **session and login security** (an expired certificate and a session cookie that isn't locked down) and it **shares more internal detail than it should** (a public admin API map and an application server reachable directly). None of it needs a rebuild — almost all of it is configuration fixes, which was reassuring to see as a beginner: most real issues aren't exotic, they're small things that got missed.

SCOPE AT A GLANCE

What I did

- ✓ Public pages only
- ✓ Port & header analysis
- ✓ OWASP ZAP passive scan

What I deliberately skipped

- ✗ Login attempts
- ✗ Exploitation
- ✗ Brute-force / DoS

Nine findings, ranked by risk

ID	FINDING	CATEGORY	RISK
F-06	Session cookie missing Secure/SameSite attributes	Session Management	HIGH
F-09	Exposed API documentation, incl. admin endpoints	API Security	HIGH
F-10	Suspected SSRF pattern (unconfirmed)	Input Validation	HIGH
F-02	Expired SSL/TLS certificate	Transport Security	MEDIUM
F-03	Application server exposed on port 8080	Attack Surface	MEDIUM
F-05	Missing standard security headers	Configuration	MEDIUM
F-07	No CSRF protection on forms	Session Management	MEDIUM
F-04	Server software version disclosed	Info Disclosure	LOW
F-08	Suspicious HTML comments in source	Info Disclosure	INFO

● High — act promptly
● Medium — schedule soon
● Low — best practice
● Informational

High-risk findings

F-06

HIGH

Session cookie isn't locked down

What I found — The session cookie is missing two standard protections (Secure and SameSite); only one of three expected safeguards is present.

Why it matters — Because the site's HTTPS is also broken (F-02), this session ID realistically travels in plain text. Anyone sharing a network with a user — public Wi-Fi being the obvious case — could potentially capture it and take over their session.

Fix — Add the missing cookie attributes, and fix F-02 so encryption actually applies.

F-09

HIGH

Full API map, including admin functions, is public

What I found — A complete technical map of the site's backend is published at a public address with no login required — including account access, fund transfers, and admin functions for adding users and changing passwords.

Why it matters — Normally an attacker spends real time mapping what an application can do. Here that work is handed over for free, with the admin functionality specifically labeled.

Fix — Move this documentation behind a login, or remove it from the public-facing environment entirely.

High & Medium-risk findings

F-10

HIGH (UNCONFIRMED)

Possible weakness in the "Server Status" feature

What I found — A feature checks whether a given server is "up" by taking a hostname as input and checking it from the server side. I noticed the pattern but deliberately didn't test it further — doing so would mean actively probing the target, past this assessment's read-only scope.

Why it matters — This pattern is one of the more serious web vulnerability classes when exploitable (it can let an attacker use the target's own server to reach internal systems). I can't confirm exploitability without testing, which I intentionally didn't do.

Fix — I'd recommend a properly scoped, authorized follow-up penetration test focused on this feature.

F-02

MEDIUM

SSL certificate expired and untrusted

What I found — The site's security certificate expired roughly three months before I ran this assessment. Loading it over HTTPS in a real browser confirms it's worse than just expired — Chrome blocks the connection outright with `NET::ERR_CERT_AUTHORITY_INVALID`, meaning the certificate chain doesn't lead to a trusted authority either.

Why it matters — Visitors get a broken/blocked connection instead of a secure one. For a banking-styled brand this is a bad trust signal, and it trains users to click through security warnings.

Fix — Replace the certificate with one from a trusted CA and set up automatic renewal so it doesn't get forgotten again.

Medium-risk findings

F-03

MEDIUM

Backend application server directly reachable

What I found — The application server itself answers directly on an extra network port (8080), not just the normal website address.

Why it matters — Any protections normally sitting in front of the main website could potentially be routed around by going straight to this extra door.

Fix — Close public access to this port; the app server should only be reachable through the normal, protected front door.

F-05

MEDIUM

Missing standard browser security protections

What I found — The site doesn't send several standard instructions that tell browsers how to protect visitors, including protection against the page being secretly embedded elsewhere (clickjacking) and against malicious scripts.

Why it matters — These are seatbelt-style protections that cost nothing to have. The clickjacking gap stood out to me especially, given this is a banking-styled site.

Fix — A one-time configuration change on the web server — standard practice for any modern site.

Medium, Low & Informational findings

F-07

MEDIUM

No protection against forged form submissions

What I found — Forms (login, feedback, etc.) don't include a hidden safety token proving a submission really came from the site itself.

Why it matters — Combined with F-06, an attacker's page could potentially trick a logged-in visitor's browser into submitting a form on their behalf, unnoticed.

Fix — Add this safety token to forms; fix F-06's cookie settings as a second layer.

F-04

LOW

Server software version publicly disclosed

What I found — Response headers name the exact backend software in use.

Fix — Configure the server to stop announcing this detail.

F-08

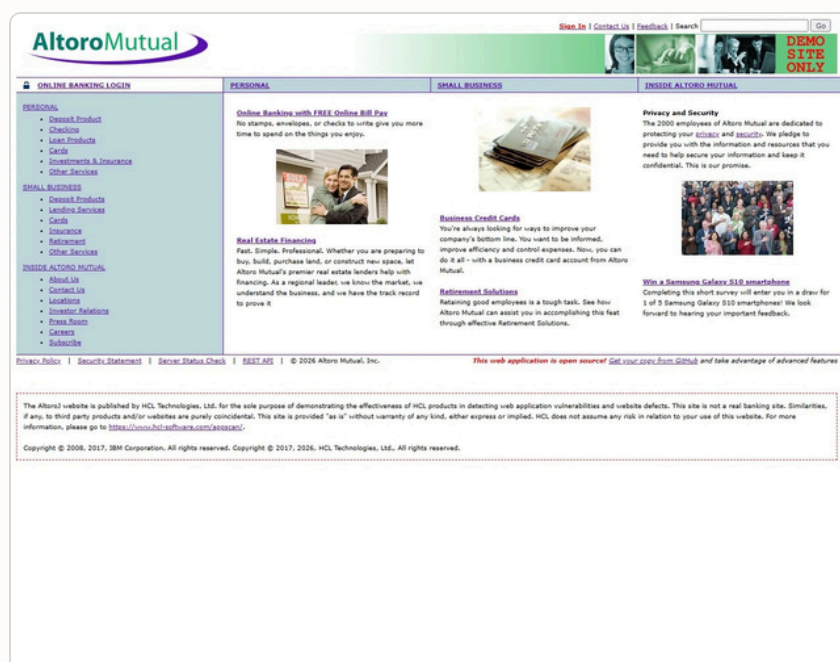
INFO

Developer comments left in page source

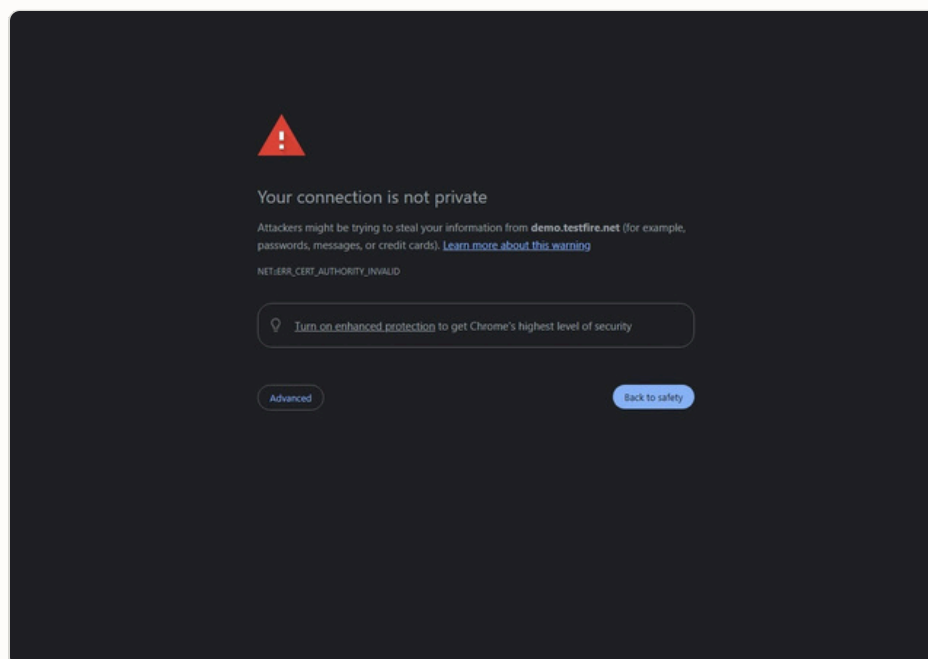
What I found — My ZAP scan flagged HTML comments across the site worth a manual look.

Fix — Review and strip unnecessary comments from what's sent to visitors' browsers.

What I actually saw

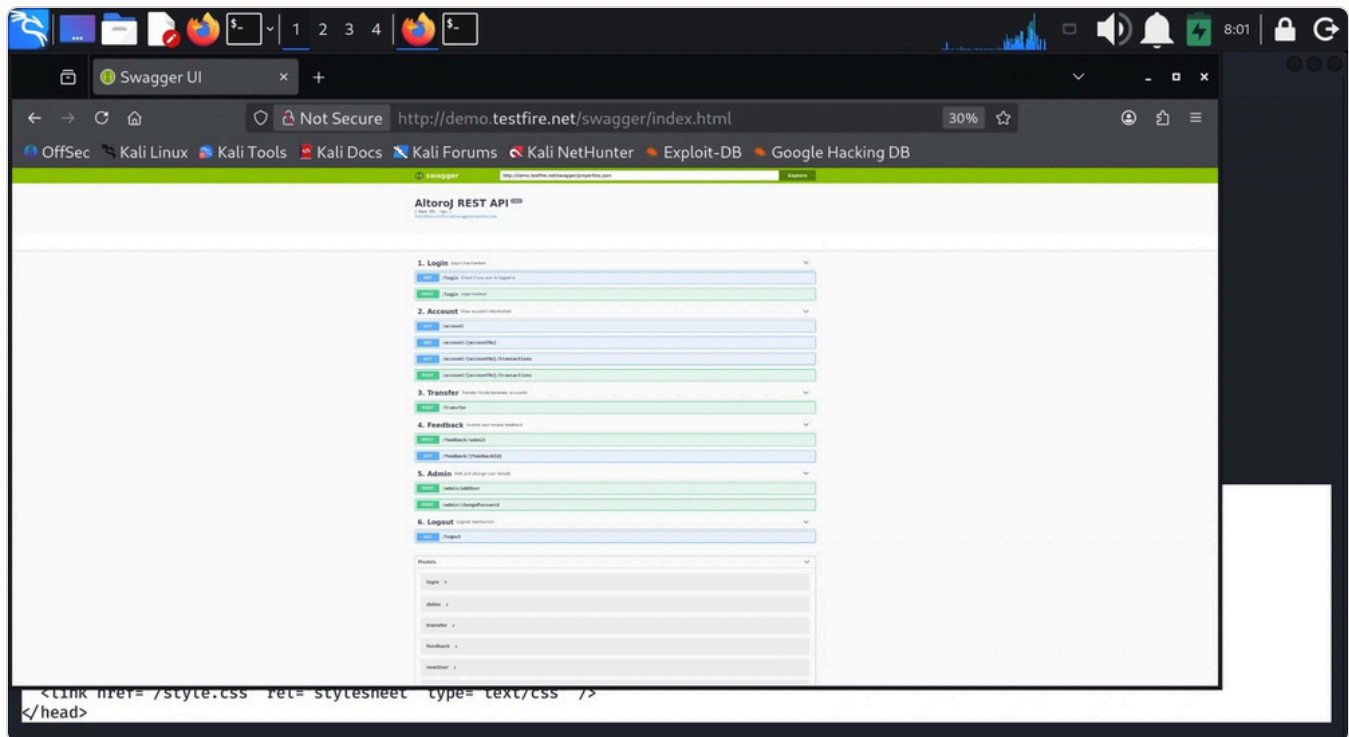


Target site homepage — demo.testfire.net, the site tested for this assessment.



F-02 — HTTPS connection blocked. Loading the site over HTTPS throws NET::ERR_CERT_AUTHORITY_INVALID: the certificate is expired and doesn't chain to a trusted authority.

The exposed API surface



F-09 — Swagger UI at /swagger/index.html. Publicly viewable with no login, listing every endpoint including **POST /admin/addUser** and **POST /admin/changePassword**.

Raw tool output (Nmap scans, HTTP headers, OWASP ZAP passive-scan alerts and crawl history) is documented in full in the accompanying GitHub repository's evidence/ folder, referenced by finding ID throughout this report.

What to fix, in order

DO FIRST

- ☐ Renew SSL certificate (F-02)
- ☐ Add missing cookie attributes (F-06)
- ☐ Add security headers (F-05)
- ☐ Restrict exposed port 8080 (F-03)

DO NEXT

- ☐ Add CSRF tokens to forms (F-07)
- ☐ Restrict API docs access (F-09)
- ☐ Suppress server version header (F-04)

PROCESS

- ☐ Automate cert renewal
- ☐ Clean up source comments (F-08)
- ☐ Scope a pentest for F-10

CONCLUSION

None of what I found here needs a rebuild — most of it is configuration changes a dev or ops team could realistically clear in a single sprint. If I had to pick where to start, I'd say the session cookie/certificate pair (F-06/F-02) and the exposed API map (F-09) matter most, since together they weaken the site's core promise of a secure login and remove an attacker's need to guess where the sensitive functionality even lives. I'd prioritize those first, then follow up with a properly scoped test on the status-check feature (F-10) to settle the one thing this passive review could flag but not confirm on its own.

This was a genuinely useful first assessment to run — it made clear how much can be learned about a site just by looking, without touching anything.